UNISAGRADO
Ciência da Computação
Testes e Qualidade de Software

Relatório técnico completo do EduTask Web

Projeto Integrado P2 | Turma A | Grupo 01

Professor Prof Dr Elvio Gilberto da Silva

Gabriel Henrique Moreira
João Lucas de Antonio Boconcelo
Lucas Braga

Bauru SP
11 de novembro de 2026

Planejamento, execução, automação e avaliação de qualidade com evidências locais e remotas. Versão revisada v1.0.1-p2. Os resultados e seus limites estão descritos nas seções e apêndices.

Sumário

Sumário 2
Síntese dos resultados 5
1 Introdução 6
1.1 Contexto e problema orientador 6
1.2 Objetivo geral e objetivos específicos 6
1.3 Identificação e contexto da revisão 6
2 Caracterização e arquitetura do sistema 6
2.1 Finalidade e fluxos funcionais 6
2.2 Arquitetura e responsabilidades 7
2.3 Modelo de dados e integridade 7
2.4 Sessão e segurança observada 8
3 Escopo requisitos e riscos 8
3.1 Escopo incluído e excluído 8
3.2 Catálogo de requisitos verificáveis 8
RF-01 8
RF-02 8
RF-03 8
RF-04 8
RF-05 9
RF-06 9
RNF-01 9
RNF-02 9
3.3 Matriz de riscos 9
4 Planejamento dos testes 10
4.1 Estratégia e níveis 10
4.2 Ambiente e dados 11
4.3 Critérios de entrada saída suspensão e retomada 11
4.4 Responsabilidades e acompanhamento 11
5 Técnicas e especificação dos casos 11
5.1 Particionamento de equivalência 11
5.2 Análise de valores limite 11
5.3 Tabela de decisão e transições 11
5.4 Rastreabilidade e referência ao catálogo 12
6 Execução pela interface e defeitos 12
6.1 Procedimento da inspeção interativa 12
6.2 Diferenças entre validação do navegador e do servidor 13
6.3 Registro e correção dos defeitos 13

7 Automação e reprodutibilidade 14
7.1 Inventário dos cenários 14
7.2 Qualidade das asserções 14
7.3 Instruções principais 14
8 Integração API e sistema 15
8.1 Contratos da API 15
8.2 Fluxos de sistema 15
9 Aceitação e verificações não funcionais 15
9.1 Critérios de aceitação observados 15
9.2 Desempenho TNF 01 e TNF 02 16
9.3 Usabilidade TNF 03 e TNF 04 16
9.4 Segurança TNF 05 e TNF 06 16
10 Métricas indicadores e análise estática 17
10.1 Indicadores obrigatórios 17
10.2 Cobertura de código e seus limites 17
10.3 Análise estática antes e depois 17
11 TDD integração contínua e melhoria 18
11.1 Experiência TDD com commits cronológicos reais 18
11.2 Pipeline real no GitHub Actions 19
11.3 Plano de melhoria 20
12 Discussão dos resultados e riscos residuais 20
13 Conclusão e parecer de qualidade 20
Referências 21
Apêndice A Catálogo completo dos 24 casos 22
Caso CT-001 Cadastro de usuário com dados válidos e senha forte 22
Caso CT-002 Cadastro de usuário com e-mail em formato inválido 22
Caso CT-003 Cadastro com senha de 7 caracteres (Limite inferior inválido) 22
Caso CT-004 Cadastro com senha sem caractere especial 23
Caso CT-005 Cadastro com e-mail já existente no banco de dados 23
Caso CT-006 Cadastro com senhas divergentes na confirmação 23
Caso CT-007 Autenticação de usuário com credenciais válidas 24
Caso CT-008 Autenticação com senha incorreta 24
Caso CT-009 Autenticação com e-mail não existente 25
Caso CT-010 Tentativa de acesso direto a rotas restritas sem sessão autenticada 25
Caso CT-011 Cadastro de disciplina com dados válidos e código padrão 25
Caso CT-012 Cadastro de disciplina com código fora do padrão formatado 26
Caso CT-013 Cadastro de disciplina com carga horária no limite inferior válido (15h) 26
Caso CT-014 Cadastro de disciplina com carga horária abaixo do limite mínimo (14h) 26
Caso CT-015 Cadastro de disciplina com carga horária no limite superior válido (400h) 27

Caso CT-016 Cadastro de disciplina com carga horária acima do limite máximo (401h) 27
Caso CT-017 Tentativa de cadastrar disciplina com código duplicado 27
Caso CT-018 Tentativa de exclusão de disciplina com tarefas vinculadas 28
Caso CT-019 Criação de atividade com dados válidos e data futura 28
Caso CT-020 Criação de atividade com título de 2 caracteres (Limite inferior inválido) 28
Caso CT-021 Criação de atividade sem informar data limite de entrega 29
Caso CT-022 Atualização de status da tarefa e recálculo da taxa de conclusão no Dashboard 29
Caso CT-023 Filtragem combinada por status "Pendente" e prioridade "Alta" 29
Caso CT-024 Busca textual por termo inexistente 30
Apêndice B Registros detalhados dos defeitos 31
DEF-03 Taxa de conclusão aceita concluídas positivas com total zero 31
DEF-04 Configuração de banco ignorada e testes contaminam demonstração 31
DEF-05 API permite leitura e alteração sem sessão e mistura dados de usuários 31
DEF-06 Data inválida na API causa exceção em vez de resposta 400 31
DEF-07 API recebe JSON ou credenciais de tipos inválidos 32
DEF-08 Hash divergente do RNF-02 original 32
Apêndice C Evidências visuais selecionadas 33
Apêndice D Instalação execução e organização da entrega 35
Ambiente e versões verificadas 35
Instalação configuração e execução 35
Comandos de verificação 35
Organização final das pastas 35
Evidências e integridade 36
Confirmações finais 36

Síntese dos resultados

O Projeto Integrado P2 aplicou planejamento, especificação, execução, automação e análise de qualidade ao EduTask Web. A versão revisada v1.0.1-p2 possui código executável, banco SQLite, interface web, API e regras de domínio separadas. A entrega reúne documentos rastreáveis, arquivos-fonte e evidências que permitem repetir a avaliação.

Foram aprovados **145 cenários automatizados**: 34 unitários, 79 de integração/API, 24 de reexecução do catálogo e quatro de aceitação técnica, além de quatro fluxos Selenium incluídos nesse total. A cobertura da suíte sem navegador é de **98.32% das linhas executáveis**. O Flake8 passou de 179 ocorrências na linha de base para zero depois da revisão.

Além da suíte, foram realizadas **28 observações interativas pela interface**, sendo 24 casos do catálogo e quatro cenários de aceitação técnica. As capturas e os snapshots foram preservados. Essas observações não são contadas novamente como testes da suíte e não constituem assinatura ou homologação humana.

Seis defeitos confirmados foram corrigidos e registrados nas issues #1, #2, #3, #4, #6 e #7. A revisão publicada passou no GitHub Actions. Um PR separado demonstrou falha intencional detectada pelo pipeline e uma execução aprovada depois da remoção da asserção falsa. O experimento foi encerrado sem merge na versão principal.

O parecer é favorável ao uso no laboratório acadêmico dentro do escopo verificado. Permanecem dependentes das pessoas responsáveis a aprovação docente, a conferência de participação e assinaturas, a homologação humana quando exigida e a entrega na pasta oficial do Drive.

1 Introdução

1.1 Contexto e problema orientador

Estudantes precisam organizar disciplinas, tarefas, prazos e prioridades. Uma aplicação que apresenta dados incorretos, permite acesso indevido ou aceita entradas inconsistentes pode prejudicar esse acompanhamento. O EduTask Web foi usado como objeto acadêmico para observar esses riscos e aplicar as técnicas da disciplina Testes e Qualidade de Software.

O problema orientador é determinar, com verificações reproduzíveis, quais requisitos selecionados são atendidos, quais falhas foram encontradas e quais limitações continuam relevantes. A avaliação não se baseia apenas em o sistema abrir no navegador: exige confrontar o esperado com o observado e conservar o vínculo entre requisito, risco, caso, resultado e defeito.

1.2 Objetivo geral e objetivos específicos

O objetivo geral é planejar, executar e avaliar a qualidade do EduTask Web em ambiente acadêmico controlado. Os objetivos específicos são delimitar requisitos e riscos; projetar casos com equivalência, limites e decisões; verificar comportamento e persistência; registrar defeitos; automatizar níveis complementares; medir indicadores; experimentar TDD; e comprovar a execução contínua no GitHub.

A estrutura segue os produtos P01 a P14 e as quantidades mínimas do manual fornecido pelo professor. Cada conjunto de evidências possui um papel definido. O relatório interpreta os resultados; ele não substitui os scripts, dados brutos, capturas ou instruções de reprodução.

1.3 Identificação e contexto da revisão

Instituição: UNISAGRADO. Curso: Ciência da Computação. Disciplina: Testes e Qualidade de Software. Professor: Prof. Dr. Elvio Gilberto da Silva. Turma A, Grupo 01. Integrantes: Gabriel Henrique Moreira, João Lucas de Antonio Boconcelo e Lucas Braga.

Versão revisada: v1.0.1-p2, identificação documental sem tag de release criada. Data de consolidação: 11/11/2026. Repositório: https://github.com/joaoboconcelo/edutask-tqs-p2 . A revisão técnica inicial foi publicada no commit `33d441c7f6dc4599da735727f232c5c1aa5d41f3`; alterações documentais posteriores estão identificadas no histórico.

Apresentação e vídeo foram dispensados conforme informação do solicitante. A estrutura da pasta 14 foi mantida para respeitar a organização do manual. A confirmação docente da dispensa e a eventual redistribuição da pontuação não foram presumidas.

2 Caracterização e arquitetura do sistema

2.1 Finalidade e fluxos funcionais

O EduTask Web permite cadastrar uma conta, autenticar um estudante, registrar disciplinas, criar tarefas vinculadas, atualizar status e consultar indicadores. Os quatro fluxos centrais usados na avaliação são autenticação/cadastro; manutenção de disciplinas; criação e acompanhamento de tarefas; e dashboard com filtros e percentual de conclusão.

O formulário de usuário exige nome, e-mail válido e confirmação de senha. O formulário de disciplina exige código no formato definido, nome, professor e carga entre 15h e 400h. O formulário de tarefa exige título de pelo menos três caracteres, disciplina existente, prazo e prioridade. A interface aceita uma tarefa vencida com aviso; sua urgência é calculada pela data, não pelo valor de prioridade escolhido.

2.2 Arquitetura e responsabilidades

Camada	Arquivos principais	Responsabilidade
Inicialização	`edutask/app.py`, `run.py`	Criar aplicação, aplicar configuração, inicializar dados e registrar blueprints
Apresentação	`edutask/templates/*.html`	Renderizar formulários, alertas, filtros, listas e dashboard
Controle web	`routes/auth_routes.py`, `discipline_routes.py`, `task_routes.py`, `main_routes.py`	Interpretar formulários, consultar sessão e acionar persistência e domínio
API	`routes/api_routes.py`	Expor recursos JSON, exigir sessão e responder com status HTTP
Domínio	`services/task_service.py`	Validar entradas, calcular percentual e urgência, filtrar tarefas
Persistência	`edutask/models.py`	Abrir SQLite, criar tabelas e carregar dados fictícios

O fluxo típico de uma inclusão é: o navegador envia um formulário; a rota verifica a sessão e a entrada; as regras de domínio validam os valores; uma consulta parametrizada grava no banco; a rota redireciona com feedback; e a tela seguinte consulta e exibe os dados. Na API, os resultados são retornados em JSON, com os códigos HTTP correspondentes.

Essa separação permite testar funções de domínio sem navegador, rotas com persistência integrada e fluxos completos com Selenium ou inspeção interativa. Uma aprovação em um nível não substitui os demais: funções corretas podem estar ligadas a rotas incorretas, e respostas de API corretas não demonstram, por si só, a clareza da interface.

2.3 Modelo de dados e integridade

Tabela	Campos e regras relevantes	Relações
`users`	id, name, email único, password_hash, role, created_at	Um usuário possui disciplinas e tarefas
`disciplines`	id, code único, name, teacher, workload_hours, user_id	Referencia users; pode ser referenciada por tarefas
`tasks`	id, title, description, due_date, priority, status, discipline_id, user_id, urgency_score, urgency_level, created_at	Referencia disciplina e usuário

Os valores de status são Pendente, Em Andamento e Concluída. As prioridades são Baixa, Média e Alta. Restrições CHECK reforçam esses domínios no banco. A conexão ativa chaves estrangeiras com PRAGMA foreign_keys=ON. A aplicação bloqueia a exclusão de uma disciplina com tarefas vinculadas.

Os códigos de disciplinas permanecem únicos globalmente, conforme o esquema atual; não são apenas únicos por usuário. A revisão restringiu consultas e alterações ao proprietário dos dados. A configuração DATABASE é lida por instância da aplicação, permitindo que dois testes ou aplicações tenham bancos diferentes sem compartilhar registros.

2.4 Sessão e segurança observada

A autenticação verifica a senha contra o hash armazenado. A sessão guarda a identificação do usuário autenticado. Rotas privadas da interface redirecionam ao login, e endpoints da API sem sessão retornam 401. Recursos pertencentes a outra pessoa não são retornados ou alterados pelas operações cobertas nos testes.

As consultas usam parâmetros, sem concatenar diretamente a entrada do usuário em valores SQL. A variável SECRET_KEY deve ser configurada para qualquer uso além da demonstração. O servidor de desenvolvimento e as credenciais fictícias não representam uma configuração de produção pública.

3 Escopo requisitos e riscos

3.1 Escopo incluído e excluído

Foram incluídos os módulos de usuários, disciplinas, tarefas, urgência, dashboard, filtros e API. Foram excluídos pagamentos, envio real de e-mails, aplicações móveis nativas, integrações institucionais reais, testes destrutivos em terceiros e uma auditoria completa de segurança. O ambiente utiliza apenas dados fictícios e execução local ou no runner de CI.

3.2 Catálogo de requisitos verificáveis

RF-01

O sistema deve permitir o cadastro de novos usuários com nome, e-mail válido e senha segura (mínimo 8 caracteres, 1 maiúscula, 1 número e 1 caractere especial).

Tipo: Funcional. Prioridade: Alta.

Critério de aceitação: **Dado** que o usuário preenche o formulário de cadastro com dados válidos e senha forte, **Quando** submeter o formulário, **Então** o sistema grava o usuário com senha criptografada e redireciona para a tela de login com mensagem de sucesso.

RF-02

O sistema deve autenticar o usuário validando as credenciais cadastradas e impedindo o acesso a páginas privadas sem sessão ativa.

Tipo: Funcional. Prioridade: Alta.

Critério de aceitação: **Dado** que o usuário informa e-mail e senha cadastrados, **Quando** clicar em "Entrar", **Então** o sistema inicia a sessão de navegação e exibe o Dashboard principal. Caso as credenciais estejam incorretas, exibe mensagem de erro sem expor detalhes internos.

RF-03

O sistema deve permitir o cadastro de disciplinas com código único formatado (`[A-Z]{2,4}-[A-Z0-9]{2,6}`) e carga horária entre 15h e 400h.

Tipo: Funcional. Prioridade: Alta.

Critério de aceitação: **Dado** que o usuário preenche código da disciplina no formato válido e carga horária entre 15h e 400h, **Quando** salvar a disciplina, **Então** o sistema insere o registro no banco. Se o código já existir, rejeita a inserção informando duplicidade.

RF-04

O sistema deve permitir a criação de atividades acadêmicas vinculadas a uma disciplina existente, exigindo título, data limite de entrega e prioridade.

Tipo: Funcional. Prioridade: Alta.

Critério de aceitação: **Dado** que o usuário seleciona uma disciplina e preenche título (mín. 3 caracteres) e data limite, **Quando** clicar em "Salvar Atividade", **Então** a tarefa é criada com status inicial "Pendente" e listada nas consultas.

RF-05

O sistema deve permitir a atualização do status da tarefa (`Pendente`, `Em Andamento`, `Concluída`) e recalcular automaticamente a taxa percentual de conclusão no painel.

Tipo: Funcional. Prioridade: Média.

Critério de aceitação: **Dado** que uma tarefa existente está com status "Pendente", **Quando** o usuário alterar seu status para "Concluída", **Então** a alteração é persistida e a taxa de conclusão no Dashboard é atualizada imediatamente.

RF-06

O sistema deve disponibilizar busca textual e filtros dinâmicos de tarefas combináveis por status e prioridade.

Tipo: Funcional. Prioridade: Média.

Critério de aceitação: **Dado** que existem múltiplas tarefas cadastradas, **Quando** o usuário selecionar o filtro de status "Pendente" e prioridade "Alta", **Então** apenas as tarefas que satisfazem simultaneamente ambos os critérios devem ser exibidas.

RNF-01

Desempenho e Tempo de Resposta: As operações de consulta e retorno da API REST devem responder em tempo inferior a 500 milissegundos em ambiente local controlado.

Tipo: Não Funcional. Prioridade: Alta.

Critério de aceitação: **Dado** que o servidor está em execução sob carga padrão de testes, **Quando** for realizada uma requisição `GET /api/tarefas` ou `GET /api/metricas`, **Então** o tempo de resposta medido deve ser estritamente menor que 500ms.

RNF-02

Segurança e Proteção de Senhas: Todas as senhas dos usuários devem ser armazenadas exclusivamente em formato de hash criptográfico seguro (PBKDF2-SHA256), sem jamais persistir senhas em texto puro.

Tipo: Não Funcional. Prioridade: Alta.

Critério de aceitação: **Dado** que um usuário efetua seu cadastro, **Quando** o registro for gravado no banco SQLite, **Então** a coluna `password_hash` deve conter um hash criptográfico válido e a senha em texto puro não deve ser identificável.

RF-05 mantém seu significado de atualização de status e percentual. O cálculo de urgência complementa RF-04 e mitiga R-07. RNF-02 mantém PBKDF2-SHA256 conforme a especificação original. O cadastro e o seed usam o método explicitamente; hashes antigos são atualizados após login válido, sem alterar a senha. A migração foi verificada nos caminhos web e API.

3.3 Matriz de riscos

Risco	Descrição	Probabilidade e impacto	Prioridade
R-01	**Vazamento ou armazenamento inseguro de senhas em texto plano no banco de dados.**	Baixo/Alto	**Alta**

Risco	Descrição	Probabilidade e impacto	Prioridade
R-02	**Acesso indevido a rotas e dados privados sem autenticação prévia de sessão.**	Médio/Alto	**Alta**
R-03	**Injeção de códigos ou dados inconsistentes em campos de cadastro (ex: e-mails malformados e senhas fracas).**	Alto/Médio	**Alta**
R-04	**Duplicidade de códigos de disciplinas gerando ambiguidade no histórico escolar.**	Médio/Médio	**Média**
R-05	**Inconsistência de integridade referencial ao excluir disciplina que possui tarefas dependentes.**	Médio/Alto	**Alta**
R-06	**Cálculo errôneo da taxa de conclusão no Dashboard quando o total de tarefas for igual a zero (Divisão por zero).**	Médio/Alto	**Alta**
R-07	**Inconsistência no cálculo de urgência de tarefas com prazos vencidos ou datas malformadas.**	Médio/Médio	**Média**
R-08	**Degradação de desempenho ou lentidão nas consultas da API REST.**	Baixo/Médio	**Baixa**

A escala é qualitativa, conforme o documento P02, sem inventar pontuações numéricas. Armazenamento inseguro é verificado pelo hash após cadastro; acesso indevido é confrontado com requisições sem sessão e de outro usuário; entradas inválidas são tratadas com classes e limites; duplicidade é exercitada com código existente; vínculos são testados na exclusão; contagens inconsistentes são rejeitadas; urgência usa fronteiras de datas; e desempenho é medido por requisições repetidas.

4 Planejamento dos testes

4.1 Estratégia e níveis

Os testes unitários verificam regras puras com entradas pequenas e determinísticas. A integração exercita o cliente Flask, as rotas e um banco SQLite real em arquivo temporário. O sistema é verificado com quatro fluxos Selenium, incluindo navegação, formulários e resultados visíveis. Os cenários de aceitação técnica verificam objetivos do estudante; a inspeção interativa complementa as asserções com leitura do feedback e capturas.

O catálogo de 24 casos foi integralmente selecionado para reexecução automatizada nesta revisão. Por isso, a taxa de automação do catálogo usa 24 como denominador. O mínimo obrigatório de 15 testes automatizados do manual não foi tratado como se fosse um plano do grupo para produzir artificialmente uma taxa de 120%.

4.2 Ambiente e dados

O ambiente local verificado usa Python 3.13.2 e Windows 11, com Flask e SQLite. O runner remoto utiliza Ubuntu e Python 3.13, conforme o workflow. O navegador local e suas capacidades estão registrados em `09_SISTEMA_INTERFACE/evidencias/navegador.json`; os testes remotos preservam seus próprios artefatos.

Cada teste integrado recebe banco temporário, com um usuário de demonstração, duas disciplinas e três tarefas iniciais. O Selenium cria servidor próprio em porta livre e não exige um processo na porta 5000. A inspeção interativa usou outra instância descartável, na porta 5011, acumulando apenas seus dados de cenário. Nenhum desses procedimentos exige apagar o banco de demonstração.

4.3 Critérios de entrada saída suspensão e retomada

Critérios de entrada: dependências instaladas, aplicação inicializável, dados fictícios e configurações de isolamento válidas. Critérios de saída técnica: cenários planejados executados, resultados aprovados, ausência de ocorrências Flake8 e latências dentro do limite na carga definida. Critérios documentais: vínculos entre requisitos e evidências, arquivos íntegros e limitações identificadas.

Interrompe-se a rodada afetada quando servidor, driver ou banco ficam indisponíveis. A causa é corrigida antes de repetir as verificações pertinentes. Durante a revisão houve incompatibilidade de driver e instabilidade de preenchimento nos testes iniciais do Selenium; a execução final utilizou driver compatível e preenchimento controlado, com duas execuções consecutivas aprovadas antes da publicação.

4.4 Responsabilidades e acompanhamento

O plano original organiza Gabriel Henrique Moreira em planejamento, João Lucas de Antonio Boconcelo em automação e Lucas Braga em análise e execução. Esses papéis não são prova de presença em cada atividade. O diário final contém os fatos verificáveis da revisão e deixa a participação individual e as horas para conferência dos próprios integrantes.

5 Técnicas e especificação dos casos

5.1 Particionamento de equivalência

Foram escolhidas classes de e-mails válidos/inválidos, senhas com e sem componentes obrigatórios, códigos no formato aceito e códigos malformados, usuários existentes/inexistentes e buscas com/sem correspondência. A estratégia reduz repetições de entradas que exercitam a mesma regra sem assumir que um único valor cobre todos os casos possíveis.

5.2 Análise de valores limite

A carga horária é verificada em 14, 15, 400 e 401 horas. A senha de sete caracteres representa a classe imediatamente abaixo do mínimo de oito. Para urgência, os testes usam datas a -1, 0, 1, 2, 3, 7 e 8 dias do dia de referência, cobrindo as transições das faixas. O percentual de conclusão verifica coleção vazia, valores normais e contagens impossíveis.

5.3 Tabela de decisão e transições

Status selecionado	Prioridade selecionada	Condição para aparecer
Pendente	Alta	Satisfazer ambos os critérios
Pendente	Baixa	Pendente e Baixa
Concluída	Alta	Concluída e Alta
Concluída	Baixa	Apenas registros com essa combinação, ou lista vazia
Todos	Todas	Aplicar somente busca textual, se informada

O teste de filtro verifica inclusive ausência de correspondências. A inspeção preparou uma tarefa Pendente/Alta antes de aplicar o filtro, evitando tratar uma lista vazia como evidência suficiente da lógica de combinação. As transições de status são observadas na tarefa e no valor matemático do dashboard.

5.4 Rastreabilidade e referência ao catálogo

O apêndice A incorpora os 24 casos detalhados. O arquivo P05 relaciona os seis requisitos funcionais, os dois não funcionais e os oito riscos aos seus testes e fontes de resultado. O catálogo apresenta responsáveis planejados; o registro de execução identifica o executor técnico real, sem converter planejamento em participação comprovada.

6 Execução pela interface e defeitos

6.1 Procedimento da inspeção interativa

Em 11/11/2026 foram percorridos os 24 casos com navegação e preenchimento da interface. O executor foi um assistente de validação técnica a pedido do solicitante. Foram observados o resultado de cada ação, as mensagens, as listas e os percentuais. Para cada caso foram salvos PNG, snapshot textual e JSON com data, método, executor, esperado e obtido.

Essa rodada é distinta da reexecução automatizada com Flask e dos quatro fluxos Selenium. Ela não é atribuída a um estudante e não representa homologação humana. Os relatos históricos sem capturas contemporâneas não foram somados às métricas confirmadas; a entrega final usa o registro atual verificável.

Caso	Resultado observado na interface
CT-001	Cadastro aceito e redirecionado ao login com mensagem de sucesso.
CT-002	O navegador bloqueou o envio: e-mail sem @. Validação nativa, anterior ao backend.
CT-003	Senha de sete caracteres rejeitada com mensagem específica.
CT-004	Senha sem símbolo rejeitada pelo backend.
CT-005	Cadastro duplicado rejeitado com aviso de e-mail existente.
CT-006	Confirmação divergente rejeitada.
CT-007	Login válido abriu o painel com três tarefas e taxa inicial de 33,33%.
CT-008	Senha incorreta rejeitada sem acesso ao painel.
CT-009	Usuário inexistente recebeu a mesma mensagem genérica do erro de senha.
CT-010	Acesso direto à rota privada redirecionou ao login.
CT-011	Disciplina CC-ES03 persistida e exibida com 80h.
CT-012	Código sem hífen rejeitado com orientação do formato.
CT-013	Carga mínima de 15h aceita e exibida na lista.
CT-014	O navegador bloqueou 14h e exibiu: O valor deve ser maior ou igual a 15.
CT-015	Carga máxima de 400h aceita e exibida.

Caso	Resultado observado na interface
CT-016	O navegador bloqueou 401h e exibiu: O valor deve ser menor ou igual a 400.
CT-017	Código CC-TQS01 duplicado rejeitado.
CT-018	Exclusão da disciplina CC-TQS01 bloqueada; registro permaneceu na lista.
CT-019	Tarefa criada com prazo 30/11/2026, prioridade Alta e status Pendente.
CT-020	Título AB rejeitado com mensagem de tamanho mínimo.
CT-021	Envio bloqueado pelo navegador porque a data obrigatória está vazia; valueMissing=true.
CT-022	Após concluir a tarefa, painel exibiu 2 concluídas de 4 tarefas: 50,0%.
CT-023	Filtro Pendente + Alta exibiu apenas o alvo; excluiu tarefas concluídas e de prioridade Média.
CT-024	Busca por termo inexistente exibiu estado vazio sem erro.

6.2 Diferenças entre validação do navegador e do servidor

Em CT-002, CT-014, CT-016 e CT-021 o navegador impediu o envio antes do backend, por e-mail sem @, carga abaixo de 15, carga acima de 400 e data obrigatória vazia. O resultado funcional foi atendido, mas o feedback é nativo e não a mensagem flash do servidor. O catálogo foi esclarecido para representar essa diferença real.

A reexecução por cliente Flask não aplica as restrições HTML e por isso continua necessária: ela confirma que os mesmos valores inválidos também são rejeitados no backend. A combinação das duas verificações demonstra a defesa nas duas camadas sem confundir seus mecanismos.

6.3 Registro e correção dos defeitos

Os seis defeitos foram registrados depois de observados na revisão. Os quatro primeiros foram corrigidos na rodada inicial; DEF-07 e DEF-08 foram corrigidos na rodada complementar. A linha de base de referência é o commit `a0df9e6a2dd69687bc88222fca707bdc6f877583`. A correção foi publicada no commit `33d441c7f6dc4599da735727f232c5c1aa5d41f3`. As quatro primeiras issues foram encerradas após aprovação remota. As duas complementares foram encerradas após correção publicada no commit 4acc9d5, aprovação local dos 145 cenários e reteste remoto aprovado na execução 43099616601. Os metadados ficam na pasta 06.

Defeito	Severidade	Correção	Issue
DEF-03 Contagem inconsistente com total zero	Média	Validar concluídas maior que total antes de tratar coleção vazia	#1
DEF-04 Configuração de banco ignorada	Média	Ler DATABASE por instância e inicializar em contexto	#2
DEF-05 API e dados sem isolamento adequado	Alta	Exigir sessão e restringir consultas e alterações ao proprietário	#3
DEF-06 Data inválida provoca exceção	Alta	Responder HTTP 400 sem persistência	#4

Defeito	Severidade	Correção	Issue
DEF-07 JSON ou credenciais com tipo inválido	Média	Validar contrato antes de acessar campos e responder 400	#6
DEF-08 Hash divergente do RNF-02 original	Média	PBKDF2 explícito e migração transparente após login válido	#7

Os detalhes reproduzíveis estão no apêndice B e no P07. Severidade descreve o impacto; prioridade orienta a ordem de correção. O registro local conserva o identificador DEF, enquanto o número da ferramenta é o número da issue. Os dois não são intercambiáveis.

7 Automação e reprodutibilidade

7.1 Inventário dos cenários

Conjunto	Cenários executados	Propósito
Unidade	34	Regras de validação, cálculo, filtros, fronteiras e regressão TDD
Integração e API	79	Comunicação de rotas, domínio e banco, incluindo sessão, proprietário, tipos de JSON, migração de hash e fluxos alternativos
Reexecução do catálogo	24	Confrontar cada CT com resposta e persistência
Aceitação técnica	4	Exercitar objetivos do estudante
Sistema com Selenium	4	Completar fluxos reais de navegador com verificações
Total automatizado	145	Todos aprovados, sem casos ignorados na consolidação local

Os casos parametrizados são contados por combinação executada, não como funções independentes. A inspeção interativa de 28 cenários é um conjunto adicional de observações, sem alterar o total de testes pytest. Os XMLs permitem conferir os nomes e os resultados individuais.

7.2 Qualidade das asserções

O teste positivo de criação de disciplina exige HTTP 201; uma resposta 409 não pode mais substituir o sucesso de cadastro. Os testes negativos verificam ausência de persistência. A validação do dashboard compara o percentual exato calculado por concluídas/total. Os testes de isolamento criam dois usuários ou duas aplicações e verificam que um não lê nem altera os dados do outro.

As fronteiras de urgência usam uma data de referência fixa no teste unitário. Datas correntes são usadas apenas quando o objetivo é observar o comportamento da execução naquele dia, como na interface. Na API, urgência é recalculada ao consultar a tarefa, evitando devolver uma pontuação armazenada que já ficou desatualizada.

7.3 Instruções principais

Executar da raiz do projeto, com as dependências da pasta 01 instaladas:

```
python -m pip install -r 01_SISTEMA_ALVO/requirements.txt
python -m pytest -v
```

```
python -m flake8 01_SISTEMA_ALVO/edutask --config=01_SISTEMA_ALVO/.flake8
```

O pytest.ini limita a coleta às cinco pastas de testes e exclui diretórios de evidências e dependências. Para executar somente um nível, informar a pasta 07, 08, 09 ou 10. O arquivo de reexecução dos 24 CTs fica na pasta 05. O Chrome deve estar instalado para os testes de sistema; o Selenium Manager pode precisar de rede para obter o driver compatível na primeira execução.

8 Integração API e sistema

8.1 Contratos da API

Método e rota	Objetivo	Respostas relevantes
POST /api/auth/login	Autenticar com e-mail e senha	200 válido, 400 ausência de campos, 401 credenciais inválidas
GET /api/disciplinas	Consultar disciplinas do usuário	200 autenticado, 401 sem sessão
POST /api/disciplinas	Criar disciplina	201 válida, 400 regra inválida, 409 código duplicado
GET /api/tarefas	Consultar tarefas e filtros	200 com dados do proprietário, 401 sem sessão
POST /api/tarefas	Criar tarefa e calcular urgência	201 válida, 400 entrada inválida, 404 disciplina não acessível
GET /api/tarefas/id	Consultar uma tarefa	200 própria, 404 ausente ou de outro usuário
PUT /api/tarefas/id	Atualizar status e prioridade	200 válida, 400 valor inválido, 404 não acessível
DELETE /api/tarefas/id	Remover tarefa própria	200 removida, 404 não acessível
GET /api/metricas	Consultar contagens e percentual	200 com indicadores do usuário, 401 sem sessão

A autenticação da API usa cookies de sessão. Na coleção Postman, executar o login válido primeiro e manter os cookies. A coleção e o ambiente são arquivos JSON editáveis. A revisão não apresenta uma execução Newman como se tivesse sido realizada: o resultado comprovado de integração vem dos testes Python e do pipeline.

8.2 Fluxos de sistema

E2E-01 verifica login, painel e logout, incluindo retorno ao login ao tentar abrir uma rota privada. E2E-02 cadastra uma disciplina e confirma código e nome na tabela. E2E-03 cadastra uma tarefa e verifica o título na lista. E2E-04 conclui uma tarefa e confronta o percentual esperado com a barra do dashboard.

O servidor do Selenium utiliza porta livre e banco temporário. As esperas são explícitas, os recursos são encerrados ao final e as capturas preservam o que foi exibido. A rotina utiliza preenchimento controlado de campos e submissão do formulário; não pretende avaliar habilidade de digitação ou comportamento de tecnologia assistiva.

9 Aceitação e verificações não funcionais

9.1 Critérios de aceitação observados

Os quatro objetivos do estudante foram verificados tanto na suíte quanto na inspeção de interface. UAT-01 reaproveita o usuário cadastrado no CT-001 e confirma primeiro login e painel próprio vazio. UAT-02 cria sua disciplina. UAT-03 cria uma entrega para dois dias depois e verifica Alta Urgência. UAT-04 conclui sua única tarefa e verifica 100% de progresso.

Cenário	Resultado observado	Evidência na pasta 10
UAT-01	Novo usuário criado em CT-001 conseguiu primeiro login e recebeu painel próprio vazio, com 0 tarefas e 0%.	`evidencias_interface/UAT-01.png`
UAT-02	Estudante cadastrou disciplina de 80h e consultou apenas a sua disciplina.	`evidencias_interface/UAT-02.png`
UAT-03	Entrega para 13/11/2026, dois dias após a validação, exibiu Alta Urgência.	`evidencias_interface/UAT-03.png`
UAT-04	Única tarefa concluída: painel exibiu 1 concluída de 1 tarefa e progresso de 100,0%.	`evidencias_interface/UAT-04.png`

A decisão é técnica, baseada na correspondência entre critério e interface. O campo de homologação humana não foi preenchido em nome de um integrante. Essa distinção permite aproveitar as evidências sem inventar a presença de um representante externo ou sua assinatura.

9.2 Desempenho TNF 01 e TNF 02

O procedimento faz três requisições de aquecimento e 30 medições por rota, em HTTP real de loopback, com um cliente sequencial. A base contém um usuário, duas disciplinas e três tarefas. O cronômetro usa perf_counter_ns e converte a duração para milissegundos. Cada amostra registra rota, repetição, status HTTP, tempo e quantidade de bytes.

Rota	N	Mínimo ms	Média ms	P95 ms	Máximo ms
`/api/metricas`	30	4.098	4.572	5.146	5.250
`/api/tarefas`	30	3.193	4.206	4.147	26.937
`/tarefas`	30	3.801	4.157	4.578	5.488

Todas as amostras ficaram abaixo de 500 ms, atendendo RNF-01 na carga documentada. O P95 é o 29º valor da lista ordenada de 30 amostras, método do posto mais próximo. Essa medição não é tempo de renderização completa do navegador, não inclui internet e não demonstra capacidade sob concorrência ou volume elevado.

Reprodução: `python tools/medir_qualidade.py`. Fontes: `latencias_http.csv` e `desempenho.json` na pasta 11. O ambiente e a data de medição constam do JSON, permitindo distinguir novas medições de resultados preservados.

9.3 Usabilidade TNF 03 e TNF 04

TNF-03 inspeciona a navegação dos fluxos centrais, a disponibilidade dos controles e a capacidade de chegar aos resultados. Na instância de inspeção, o menu compacto foi expandido para acessar opções de sessão. Os fluxos foram completados e suas telas registradas. Isso é uma inspeção técnica de navegação, não uma pesquisa com amostra de usuários.

TNF-04 verifica feedback de validação: senhas curtas ou sem símbolo, divergência de confirmação, login inválido, código de disciplina fora do padrão, duplicidade, título curto, limites de carga e data obrigatória. As mensagens nativas e as mensagens flash foram distinguidas. Os alertas do template possuem role=alert; contraste e leitores de tela não foram medidos nesta revisão.

9.4 Segurança TNF 05 e TNF 06

TNF-05 cadastra um usuário, consulta password_hash em banco temporário, exige o prefixo PBKDF2-SHA256, rejeita senha em texto claro e confirma a verificação da senha. O seed segue o mesmo algoritmo, e os testes

verificam migração de hashes antigos após login válido, tanto pela interface quanto pela API. TNF-06 verifica ausência de sessão, tentativa de acesso a registros de outro usuário e entradas de SQL injection sem autenticação ou alteração indevida da base.

Os testes não constituem pentest completo. Permanecem fora da validação uma defesa CSRF dedicada, limitação de tentativas, implantação HTTPS e configurações de produção. Por esse motivo o parecer de uso se limita ao laboratório acadêmico controlado.

10 Métricas indicadores e análise estática

10.1 Indicadores obrigatórios

Indicador	Fórmula e universo	Resultado	Fonte
Percentual de execução	CTs reexecutados / CTs planejados	24/24 = 100%	catálogo e XML
Taxa de aprovação	CTs aprovados / CTs executados	24/24 = 100%	catalogo.xml
Taxa de reprovação	CTs reprovados / CTs executados	0/24 = 0%	catalogo.xml
Cobertura de requisitos	Requisitos com verificação executada / escopo	8/8 = 100%	P05, testes e latências
Taxa de automação	CTs automatizados / CTs selecionados para automação	24/24 = 100%	P03 e script da pasta 05
Defeitos por severidade	Contagem dos seis confirmados	4 médios, 2 altos	issues e P07
Defeitos por status	Contagem após reteste	6 encerrados, 0 abertos	GitHub Issues
Taxa de reabertura	Defeitos reabertos / retestados	0/6 = 0%	registros da revisão

A planilha Planilha_Metricas.xlsx contém dados identificados e 12 fórmulas, incluindo os indicadores de execução, aprovação, reprovação, requisitos, automação, defeitos, cobertura de linhas e aprovação da suíte. Os indicadores por severidade/status são contagens. Os dados históricos sem evidência não foram incorporados aos seis defeitos confirmados.

10.2 Cobertura de código e seus limites

Linhas executadas: 586/596 = **98.32%**. Ramos cobertos: 162/172 = **94.19%**. A cobertura foi medida na suíte sem Selenium; os XMLs do navegador são contabilizados separadamente.

Cobertura de requisito e cobertura de código são medidas diferentes. Um requisito com ao menos um teste não significa que todas as suas alternativas foram exercitadas. Da mesma forma, uma linha executada não prova que todos os seus resultados foram confrontados com uma asserção. O JSON de cobertura preserva informações por arquivo e linhas não cobertas.

10.3 Análise estática antes e depois

Foram usadas a mesma configuração Flake8 e a mesma largura máxima de 120 caracteres. A linha de base apresentou 179 ocorrências; a execução após a revisão não apresentou ocorrências. Foram corrigidos espaços residuais, importações não usadas, exportações de pacote e linhas excessivamente longas.

O workflow usa o código de saída real do Flake8, sem --exit-zero. Portanto, uma ocorrência faz a etapa falhar. Não foi apresentada medição Radon ou complexidade ciclomática que não tenha sido executada; o relatório se limita aos instrumentos e dados efetivamente disponíveis.

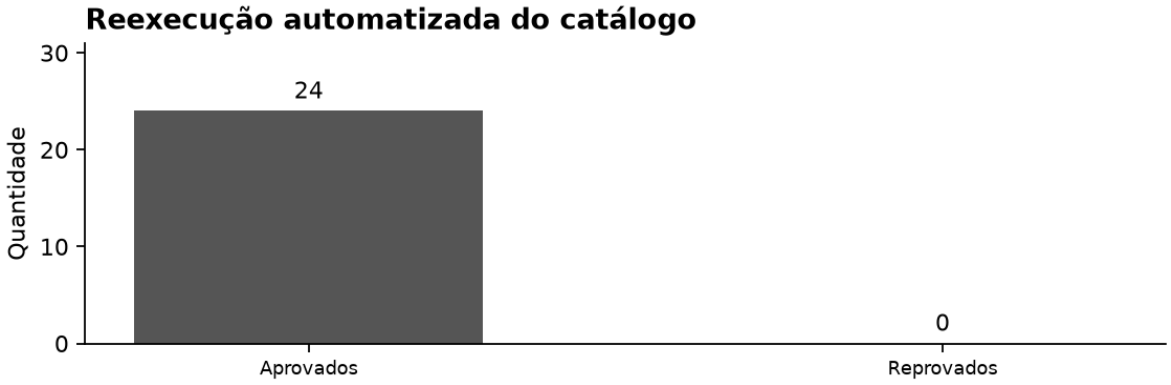


Figura 1 Reexecução dos casos do catálogo

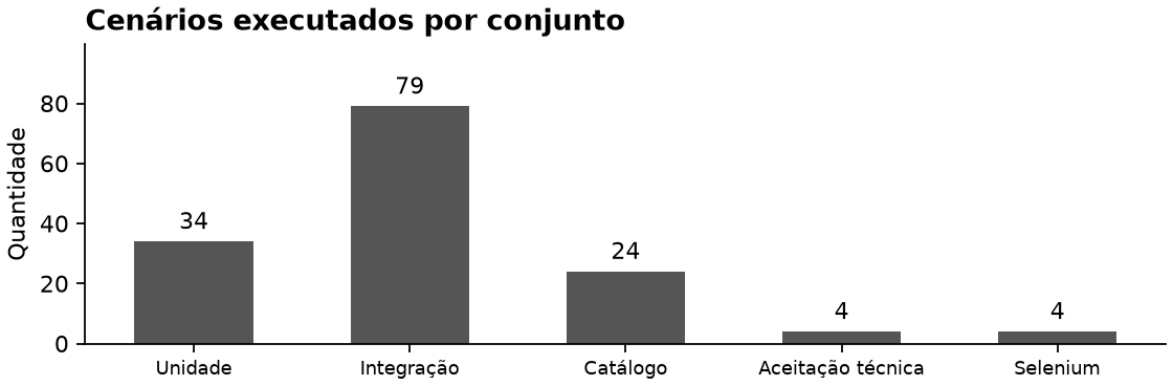


Figura 2 Distribuição dos cenários automatizados

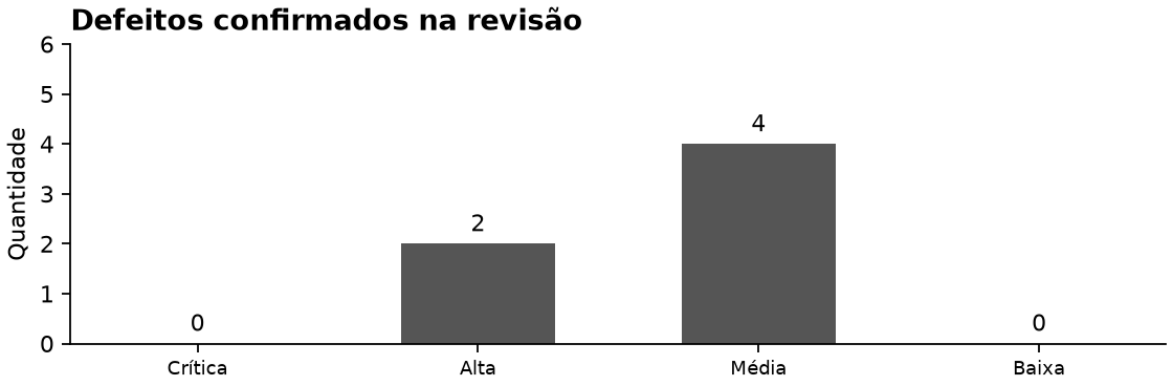


Figura 3 Severidade dos defeitos confirmados

11 TDD integração contínua e melhoria

11.1 Experiência TDD com commits cronológicos reais

O comportamento escolhido foi rejeitar corpos JSON não objeto e credenciais de tipos não textuais com resposta HTTP 400. Antes da correção, entradas como [1], texto, números e campos email/password não textuais podiam provocar exceção. Os testes de integração foram escritos e executados antes da implementação e registrados no primeiro commit do ciclo.

Etapa	Commit	Data e fuso	Resultado real
RED	e2ad848831f6	2026-11-11T17:53:29-03:00	17 falhas e 3 aprovados antes da correção

Etapa	Commit	Data e fuso	Resultado real
GREEN	9feabf493df3	2026-11-11T17:54:13-03:00	20 aprovados após implementação mínima
REFACTOR	4e55cbbaa515	2026-11-11T17:54:33-03:00	20 aprovados após isolamento da validação

RED registrou 17 falhas e três aprovações: as falhas revelaram erros do comportamento existente. GREEN implementou a guarda mínima de JSON e tipos de credenciais, aprovando os 20 cenários. REFACTOR extraiu a validação para a função pura json_payload_error, preservando as 20 aprovações. São três commits reais e sucessivos; não foram criados retrospectivamente como se fossem históricos de outubro.

Links verificáveis dos commits:

- RED: https://github.com/joaoboconcelo/edutask-tqs-p2/commit/e2ad848831f68d84b2976ca20c85c59f3123b73c
- GREEN: https://github.com/joaoboconcelo/edutask-tqs-p2/commit/9feabf493df34e27a876651d08037ac668d67a30
- REFACTOR: https://github.com/joaoboconcelo/edutask-tqs-p2/commit/4e55cbbaa515950dbdcb50b02483deb37e01d131

Os arquivos red/green/refactor.txt e .xml, além de commits.json, ficam em 12_TDD_CICD_MELHORIA/evidencias/tdd_json. A primeira experiência com cálculo de percentual continua como evidência complementar de logs/snapshots, mas a comprovação principal do requisito TDD agora é este ciclo com Git. A falha intencional de CI descrita a seguir tem outra finalidade e não é confundida com um defeito encontrado por TDD.

11.2 Pipeline real no GitHub Actions

O arquivo canônico é `.github/workflows/ci.yml`, na raiz do repositório. Os gatilhos são push e pull_request em main/develop, além de workflow_dispatch. O job faz checkout, configura Python 3.13, instala dependências, executa Flake8, unidade, integração/catálogo/aceitação, Selenium e upload dos artefatos. O resumo usa o estado real do job.

Execução	Commit	Conclusão	Run ID
principal	33d441c7f6dc	success	43094289919
falha controlada	018467182875	failure	43094396473
correcao	7407afef2a4d	success	43094602358
complementar	4acc9d5d7c1d	success	43099616601

URLs verificáveis:

- Revisão publicada aprovada: https://github.com/joaoboconcelo/edutask-tqs-p2/actions/runs/43094289919
- Falha intencional detectada: https://github.com/joaoboconcelo/edutask-tqs-p2/actions/runs/43094396473
- Correção aprovada: https://github.com/joaoboconcelo/edutask-tqs-p2/actions/runs/43094602358
- Suíte ampliada e PBKDF2 aprovados: https://github.com/joaoboconcelo/edutask-tqs-p2/actions/runs/43099616601
- PR de demonstração encerrado sem merge: https://github.com/joaoboconcelo/edutask-tqs-p2/pull/5

O teste temporário continha assert False e falhou na etapa de testes unitários, como planejado. Depois de removido, o pipeline voltou a passar. A versão principal permaneceu funcional. Os logs originais do GitHub

foram baixados e extraídos na pasta 12 para que a comprovação não dependa apenas de um ícone na interface.

11.3 Plano de melhoria

Prioridade	Ação	Critério de verificação
Alta	Conferir aprovação docente, homologação e assinaturas	Registros reais das pessoas responsáveis
Alta	Adicionar proteção CSRF antes de exposição pública	POST sem token rejeitado, regressão aprovada
Média	Paginar consultas e avaliar 500 registros	Medição por volume e comparação com a base pequena
Média	Empacotar Bootstrap e ícones localmente	Fluxos estilizados com rede externa desativada
Média	Ampliar caminhos ainda não cobertos	Novas asserções de comportamento e cobertura conferida
Baixa	Avaliar PostgreSQL conforme concorrência necessária	Ensaio comparativo antes da decisão de migração

Publicação, issues e demonstração de CI estão concluídas. Os responsáveis individuais pelas próximas atividades devem ser confirmados pelo grupo, sem atribuição de execução ou horas por inferência.

12 Discussão dos resultados e riscos residuais

Os resultados mais relevantes são a correção de falhas reais de isolamento e validação, o fortalecimento das asserções, a preservação de dados brutos e a demonstração remota de que o pipeline aceita uma versão correta e bloqueia uma regressão intencional. A inspeção da interface acrescenta observações sobre mensagens nativas e navegação que o cliente Flask não reproduz.

Os 145 cenários aprovados não provam ausência geral de falhas. A cobertura de linhas e ramos é parcial, o desempenho foi medido em carga pequena, a interface usa recursos externos por CDN e a segurança não foi auditada de forma exaustiva. Esses limites orientam o plano de melhoria e restringem o parecer ao uso acadêmico local.

O solicitante não confirmou aprovação docente ou validações humanas anteriores. Por isso, a ficha do professor, o aceite humano e as assinaturas permanecem sem preenchimento. Os registros interativos deixam explícito o executor técnico. A entrega oficial no Google Drive e a conferência de permissões e prazo também não são substituídas pela publicação no GitHub.

13 Conclusão e parecer de qualidade

A versão revisada apresenta comportamento aprovado nos cenários selecionados, rastreabilidade entre requisitos e evidências, métricas calculadas a partir de resultados preservados, defeitos acompanhados na ferramenta e execução real de CI. A estrutura final conserva as 17 pastas do manual, elimina duplicatas de relatório e mantém o material necessário para instalação, execução e avaliação.

O parecer é favorável ao uso do EduTask Web no laboratório acadêmico dentro do escopo testado. A conclusão deve ser lida com as limitações de cobertura, carga e segurança apresentadas. As confirmações humanas e a aprovação docente dependem das pessoas responsáveis; o relatório não apresenta declarações ou assinaturas em nome delas.

Referências

SILVA, Elvio Gilberto da. Testes e Qualidade de Software Projeto P2. Manual fornecido pelo professor, 2026. Critérios, estrutura e modelos das seções 9 a 16 e apêndices.

EduTask Web. Código-fonte e documentos P01 a P14 desta entrega. Linha de base a0df9e6 e revisão publicada 33d441c. Repositório: https://github.com/joaoboconcelo/edutask-tqs-p2 .

Evidências efetivamente utilizadas: relatórios JUnit XML; JSON de cobertura e ambiente; CSV de latências; logs Flake8; snapshots de TDD; capturas e metadados da inspeção interativa; issues #1, #2, #3, #4, #6 e #7; runs de CI e PR #5 indicados na seção 11.

Apêndice A Catálogo completo dos 24 casos

Os responsáveis indicados neste catálogo são papéis previstos no planejamento. Os resultados da rodada atual, método e executor estão no P06 e na seção 6. As diferenças de feedback nativo foram explicitadas após a inspeção.

Caso CT-001 Cadastro de usuário com dados válidos e senha forte

- **Requisito Relacionado:** RF-01 | **Risco:** R-03
- **Nível/Tipo:** Sistema / Funcional Positivo
- **Técnica Aplicada:** Particionamento de Equivalência (Classe Válida)
- **Prioridade:** Alta
- **Pré-condições:** O sistema está rodando e a página de registro `/register` está aberta. O e-mail informado não deve existir no banco.
- **Dados de Entrada:** Nome: "Carlos Silva", E-mail: "carlos.silva@unisagrado.edu.br", Senha: "SenhaForte@2026", Confirmar: "SenhaForte@2026"
- **Passos Numerados:**

1. Acessar a tela `/register`.

2. Preencher os campos Nome, E-mail, Senha e Confirmar Senha com os dados de entrada.

3. Clicar no botão "Cadastrar".

- **Resultado Esperado:** O sistema grava o usuário no banco de dados, exibe a mensagem de sucesso "Cadastro realizado com sucesso! Efetue login para continuar." e redireciona para `/login`.
- **Candidato à Automação:** Sim (Fluxo crítico e repetitivo em testes de regressão).
- **Responsável:** Lucas Braga

Caso CT-002 Cadastro de usuário com e-mail em formato inválido

- **Requisito Relacionado:** RF-01 | **Risco:** R-03
- **Nível/Tipo:** Sistema / Funcional Negativo
- **Técnica Aplicada:** Particionamento de Equivalência (Classe Inválida)
- **Prioridade:** Alta
- **Pré-condições:** Página `/register` aberta.
- **Dados de Entrada:** Nome: "Teste Invalido", E-mail: "email_sem_arroba.com", Senha: "SenhaForte@2026", Confirmar: "SenhaForte@2026"
- **Passos Numerados:**

1. Acessar `/register`.

2. Preencher os campos com o e-mail malformado.

3. Clicar no botão "Cadastrar".

- **Resultado Esperado:** O sistema rejeita o cadastro, permanece na página de registro e exibe a mensagem de alerta "Formato de e-mail inválido.".
- **Candidato à Automação:** Sim (Validação unitária e de interface).
- **Responsável:** Lucas Braga

Caso CT-003 Cadastro com senha de 7 caracteres (Limite inferior inválido)

- **Requisito Relacionado:** RF-01 | **Risco:** R-03
- **Nível/Tipo:** Unidade/Sistema / Limite Negativo
- **Técnica Aplicada:** Análise de Valor Limite (BVA - Valor imediatamente abaixo do mínimo de 8)
- **Prioridade:** Alta
- **Pré-condições:** Página `/register` aberta.

- **Dados de Entrada:** Nome: "Usuario Limite", E-mail: "limite7@unisagrado.edu.br", Senha: "Abc@123" (7 caracteres), Confirmar: "Abc@123"
- **Passos Numerados:**

1. Preencher os dados no formulário de cadastro com a senha de 7 caracteres.

2. Submeter o formulário.

- **Resultado Esperado:** O sistema bloqueia a submissão e exibe o erro "A senha deve conter no mínimo 8 caracteres.".
- **Candidato à Automação:** Sim (Coberto por teste unitário em `validate_password_strength`).
- **Responsável:** João Lucas de Antonio Boconcelo

Caso CT-004 Cadastro com senha sem caractere especial

- **Requisito Relacionado:** RF-01 | **Risco:** R-03
- **Nível/Tipo:** Sistema / Funcional Negativo
- **Técnica Aplicada:** Particionamento de Equivalência (Critério de complexidade ausente)
- **Prioridade:** Média
- **Pré-condições:** Aplicação e banco fictício inicializados; para CT-011 em diante, usuário de demonstração autenticado. Restabelecer a base entre cenários.
- **Dados de Entrada:** Nome: "Usuario Sem Especial", E-mail: "semespecial@unisagrado.edu.br", Senha: "SenhaForte2026", Confirmar: "SenhaForte2026"
- **Passos Numerados:**

1. Preencher os dados com a senha contendo maiúsculas, minúsculas e números, porém sem caracteres especiais.

2. Clicar em "Cadastrar".

- **Resultado Esperado:** O sistema rejeita o cadastro exibindo a mensagem "A senha deve conter pelo menos um caractere especial (@$!%*?&#...).".
- **Candidato à Automação:** Sim.
- **Responsável:** Lucas Braga

Caso CT-005 Cadastro com e-mail já existente no banco de dados

- **Requisito Relacionado:** RF-01 | **Risco:** R-03
- **Nível/Tipo:** Sistema / Exceção
- **Técnica Aplicada:** Particionamento de Equivalência (Restrição de unicidade)
- **Prioridade:** Alta
- **Pré-condições:** O e-mail `aluno@unisagrado.edu.br` já deve estar registrado no banco.
- **Dados de Entrada:** Nome: "Novo Aluno", E-mail: "aluno@unisagrado.edu.br", Senha: "SenhaForte@2026", Confirmar: "SenhaForte@2026"
- **Passos Numerados:**

1. Acessar `/register`.

2. Informar o e-mail institucional duplicado.

3. Clicar em "Cadastrar".

- **Resultado Esperado:** O sistema não realiza nova inserção e exibe a mensagem de aviso "Este e-mail já está cadastrado no sistema.".
- **Candidato à Automação:** Sim.
- **Responsável:** Gabriel Henrique Moreira

Caso CT-006 Cadastro com senhas divergentes na confirmação

- **Requisito Relacionado:** RF-01 | **Risco:** R-03

- **Nível/Tipo:** Sistema / Funcional Alternativo Negativo
- **Técnica Aplicada:** Particionamento de Equivalência
- **Prioridade:** Média
- **Pré-condições:** Aplicação e banco fictício inicializados; para CT-011 em diante, usuário de demonstração autenticado. Restabelecer a base entre cenários.
- **Dados de Entrada:** Nome: "Usuario Divergente", E-mail: "divergente@unisagrado.edu.br", Senha: "SenhaForte@1", Confirmar: "SenhaForte@2"
- **Passos Numerados:**

1. Informar senhas diferentes nos dois campos.

2. Submeter formulário.

- **Resultado Esperado:** O sistema recusa o cadastro informando "As senhas digitadas não coincidem.".
- **Candidato à Automação:** Sim.
- **Responsável:** Lucas Braga

Caso CT-007 Autenticação de usuário com credenciais válidas

- **Requisito Relacionado:** RF-02 | **Risco:** R-02
- **Nível/Tipo:** Sistema / Funcional Positivo
- **Técnica Aplicada:** Fluxo Principal (Caminho Feliz)
- **Prioridade:** Alta
- **Pré-condições:** Usuário `aluno@unisagrado.edu.br` ativo com senha `Senha@123`.
- **Dados de Entrada:** E-mail: "aluno@unisagrado.edu.br", Senha: "Senha@123"
- **Passos Numerados:**

1. Acessar a tela de login `/login`.

2. Informar e-mail e senha cadastrados.

3. Clicar em "Entrar".

- **Resultado Esperado:** A sessão é criada com sucesso, o usuário é redirecionado ao Dashboard (`/`) e visualiza a mensagem de boas-vindas com o seu nome e opções da barra de navegação.
- **Candidato à Automação:** Sim (Automatizado com Selenium e API).
- **Responsável:** João Lucas de Antonio Boconcelo

Caso CT-008 Autenticação com senha incorreta

- **Requisito Relacionado:** RF-02 | **Risco:** R-02
- **Nível/Tipo:** Sistema / Funcional Negativo
- **Técnica Aplicada:** Particionamento de Equivalência
- **Prioridade:** Alta
- **Pré-condições:** Aplicação e banco fictício inicializados; para CT-011 em diante, usuário de demonstração autenticado. Restabelecer a base entre cenários.
- **Dados de Entrada:** E-mail: "aluno@unisagrado.edu.br", Senha: "SenhaIncorreta@99"
- **Passos Numerados:**

1. Acessar `/login`.

2. Informar a senha errada.

3. Clicar em "Entrar".

- **Resultado Esperado:** O sistema rejeita o acesso e apresenta "Credenciais inválidas. Verifique seu e-mail e senha.". O usuário permanece em `/login`.
- **Candidato à Automação:** Sim.
- **Responsável:** Lucas Braga

Caso CT-009 Autenticação com e-mail não existente

- **Requisito Relacionado:** RF-02 | **Risco:** R-02
- **Nível/Tipo:** Sistema / Funcional Negativo
- **Técnica Aplicada:** Particionamento de Equivalência
- **Prioridade:** Média
- **Pré-condições:** Aplicação e banco fictício inicializados; para CT-011 em diante, usuário de demonstração autenticado. Restabelecer a base entre cenários.
- **Dados de Entrada:** E-mail: "fantasma@unisagrado.edu.br", Senha: "QualquerSenha@123"
- **Passos Numerados:**

1. Informar e-mail não cadastrado.

2. Clicar em "Entrar".

- **Resultado Esperado:** O sistema apresenta a mensagem genérica "Credenciais inválidas. Verifique seu e-mail e senha.", sem revelar se o e-mail existe ou não.
- **Candidato à Automação:** Sim.
- **Responsável:** Lucas Braga

Caso CT-010 Tentativa de acesso direto a rotas restritas sem sessão autenticada

- **Requisito Relacionado:** RF-02, RNF-02 | **Risco:** R-02
- **Nível/Tipo:** Segurança / Sistema
- **Técnica Aplicada:** Teste de Controle de Acesso / Exceção
- **Prioridade:** Alta
- **Pré-condições:** O navegador deve estar sem cookies de sessão (modo anônimo ou após logout).
- **Dados de Entrada:** URL direta: `http://127.0.0.1:5000/tarefas`
- **Passos Numerados:**

1. Abrir navegador em janela anônima.

2. Digitar diretamente a URL `/tarefas` na barra de endereços e pressionar Enter.

- **Resultado Esperado:** O sistema intercepta o acesso não autenticado, impede a visualização da tela de tarefas e redireciona o usuário para `/login` com a mensagem "Faça login para acessar suas tarefas.".
- **Candidato à Automação:** Sim (Coberto em teste de integração/API e Selenium).
- **Responsável:** João Lucas de Antonio Boconcelo

Caso CT-011 Cadastro de disciplina com dados válidos e código padrão

- **Requisito Relacionado:** RF-03 | **Risco:** R-04
- **Nível/Tipo:** Sistema / Funcional Positivo
- **Técnica Aplicada:** Particionamento de Equivalência (Classe Válida)
- **Prioridade:** Alta
- **Pré-condições:** Usuário autenticado.
- **Dados de Entrada:** Código: "CC-ES03", Nome: "Engenharia de Software", Professor: "Profa. Dra. Maria Cristina", Carga Horária: 80
- **Passos Numerados:**

1. Acessar o menu "Disciplinas" e clicar em "Cadastrar Nova Disciplina".

2. Preencher os dados informados.

3. Clicar em "Salvar Disciplina".

- **Resultado Esperado:** O registro é inserido no banco, o sistema redireciona para a lista `/disciplinas` com mensagem de sucesso e a nova matéria aparece listada na tabela.
- **Candidato à Automação:** Sim (Automatizado com Selenium).

- **Responsável:** João Lucas de Antonio Boconcelo

Caso CT-012 Cadastro de disciplina com código fora do padrão formatado

- **Requisito Relacionado:** RF-03 | **Risco:** R-03
- **Nível/Tipo:** Sistema / Funcional Negativo
- **Técnica Aplicada:** Particionamento de Equivalência (Expressão Regular Inválida)
- **Prioridade:** Média
- **Pré-condições:** Aplicação e banco fictício inicializados; para CT-011 em diante, usuário de demonstração autenticado. Restabelecer a base entre cenários.
- **Dados de Entrada:** Código: "CODIGOERRADO123", Nome: "Compiladores", Professor: "Prof. Dr. Silva", Carga Horária: 60
- **Passos Numerados:**

1. Preencher o formulário com código sem hífen ou formato irregular.

2. Clicar em "Salvar Disciplina".

- **Resultado Esperado:** O sistema rejeita o cadastro e informa: "Código de disciplina inválido. Formato esperado: CC-TQS01, ENG-101 (2 a 4 letras, hífen e 2 a 6 alfanuméricos).".
- **Candidato à Automação:** Sim (Unitário e Sistema).
- **Responsável:** Lucas Braga

Caso CT-013 Cadastro de disciplina com carga horária no limite inferior válido (15h)

- **Requisito Relacionado:** RF-03 | **Risco:** R-03
- **Nível/Tipo:** Sistema / Limite Positivo
- **Técnica Aplicada:** Análise de Valor Limite (BVA - Limite Mínimo Permitido)
- **Prioridade:** Média
- **Pré-condições:** Aplicação e banco fictício inicializados; para CT-011 em diante, usuário de demonstração autenticado. Restabelecer a base entre cenários.
- **Dados de Entrada:** Código: "CC-TOP15", Nome: "Tópicos Especiais I", Professor: "Prof. Especialista", Carga Horária: 15
- **Passos Numerados:**

1. Preencher o formulário informando exatamente 15 horas de carga horária.

2. Submeter formulário.

- **Resultado Esperado:** A disciplina é cadastrada com sucesso com carga horária registrada de 15h.
- **Candidato à Automação:** Sim.
- **Responsável:** João Lucas de Antonio Boconcelo

Caso CT-014 Cadastro de disciplina com carga horária abaixo do limite mínimo (14h)

- **Requisito Relacionado:** RF-03 | **Risco:** R-03
- **Nível/Tipo:** Sistema / Limite Negativo
- **Técnica Aplicada:** Análise de Valor Limite (BVA - Valor imediatamente abaixo do mínimo permitido)
- **Prioridade:** Média
- **Pré-condições:** Aplicação e banco fictício inicializados; para CT-011 em diante, usuário de demonstração autenticado. Restabelecer a base entre cenários.
- **Dados de Entrada:** Código: "CC-SUB14", Nome: "Seminário Curto", Professor: "Prof. Teste", Carga Horária: 14
- **Passos Numerados:**

1. Preencher o formulário com 14 horas.

2. Submeter formulário.

- **Resultado Esperado:** O cadastro é bloqueado com a mensagem "A carga horária deve ser um valor numérico entre 15 e 400 horas.".
- **Candidato à Automação:** Sim.
- **Responsável:** Lucas Braga

Caso CT-015 Cadastro de disciplina com carga horária no limite superior válido (400h)

- **Requisito Relacionado:** RF-03 | **Risco:** R-03
- **Nível/Tipo:** Sistema / Limite Positivo
- **Técnica Aplicada:** Análise de Valor Limite (BVA - Limite Máximo Permitido)
- **Prioridade:** Média
- **Pré-condições:** Aplicação e banco fictício inicializados; para CT-011 em diante, usuário de demonstração autenticado. Restabelecer a base entre cenários.
- **Dados de Entrada:** Código: "CC-EST400", Nome: "Estágio Supervisionado", Professor: "Prof. Coordenador", Carga Horária: 400
- **Passos Numerados:**

1. Preencher com exatamente 400 horas.

2. Submeter formulário.

- **Resultado Esperado:** Cadastro concluído com sucesso e exibido com 400h na listagem.
- **Candidato à Automação:** Sim.
- **Responsável:** João Lucas de Antonio Boconcelo

Caso CT-016 Cadastro de disciplina com carga horária acima do limite máximo (401h)

- **Requisito Relacionado:** RF-03 | **Risco:** R-03
- **Nível/Tipo:** Sistema / Limite Negativo
- **Técnica Aplicada:** Análise de Valor Limite (BVA - Valor imediatamente acima do máximo permitido)
- **Prioridade:** Média
- **Pré-condições:** Aplicação e banco fictício inicializados; para CT-011 em diante, usuário de demonstração autenticado. Restabelecer a base entre cenários.
- **Dados de Entrada:** Código: "CC-EXT401", Nome: "Matéria Extensa", Professor: "Prof. Teste", Carga Horária: 401
- **Passos Numerados:**

1. Preencher o formulário com 401 horas.

2. Submeter formulário.

- **Resultado Esperado:** O sistema bloqueia a gravação e apresenta o erro de faixa válida entre 15 e 400 horas.
- **Candidato à Automação:** Sim.
- **Responsável:** Lucas Braga

Caso CT-017 Tentativa de cadastrar disciplina com código duplicado

- **Requisito Relacionado:** RF-03 | **Risco:** R-04
- **Nível/Tipo:** Sistema / Exceção
- **Técnica Aplicada:** Particionamento de Equivalência (Restrição de Chave Única)
- **Prioridade:** Alta
- **Pré-condições:** Disciplina com código `CC-TQS01` já existente no banco de dados.
- **Dados de Entrada:** Código: "CC-TQS01", Nome: "Segunda Turma TQS", Professor: "Outro Docente", Carga Horária: 80
- **Passos Numerados:**

1. Acessar `/disciplinas/nova`.

2. Inserir o código `CC-TQS01` já existente.

3. Clicar em "Salvar Disciplina".

- **Resultado Esperado:** O sistema bloqueia a duplicidade e emite o aviso "Já existe uma disciplina cadastrada com o código CC-TQS01.".
- **Candidato à Automação:** Sim.
- **Responsável:** Gabriel Henrique Moreira

Caso CT-018 Tentativa de exclusão de disciplina com tarefas vinculadas

- **Requisito Relacionado:** RF-03 | **Risco:** R-05
- **Nível/Tipo:** Sistema / Integridade Relacional e Exceção
- **Técnica Aplicada:** Validação de Regra de Negócio / Integridade Estrutural
- **Prioridade:** Alta
- **Pré-condições:** A disciplina `CC-TQS01` possui atividades acadêmicas cadastradas e associadas a ela.
- **Passos Numerados:**

1. Acessar a lista `/disciplinas`.

2. Localizar a linha da disciplina `CC-TQS01`.

3. Clicar no botão "Excluir" e confirmar o modal.

- **Resultado Esperado:** O sistema impede a exclusão para não gerar registros órfãos e exibe o alerta "Não é possível excluir disciplina que possui tarefas vinculadas.".
- **Candidato à Automação:** Sim.
- **Responsável:** Gabriel Henrique Moreira

Caso CT-019 Criação de atividade com dados válidos e data futura

- **Requisito Relacionado:** RF-04 | **Risco:** R-07
- **Nível/Tipo:** Sistema / Funcional Positivo
- **Técnica Aplicada:** Fluxo Principal (Caminho Feliz)
- **Prioridade:** Alta
- **Pré-condições:** Aplicação e banco fictício inicializados; para CT-011 em diante, usuário de demonstração autenticado. Restabelecer a base entre cenários.
- **Dados de Entrada:** Título: "Implementação do Pipeline CI/CD", Descrição: "Configurar actions para build e testes automáticos", Prazo: "2026-12-10", Prioridade: "Alta", Disciplina: `CC-TQS01`
- **Passos Numerados:**

1. Acessar "Tarefas" -> "Cadastrar Nova Tarefa".

2. Preencher título, descrição, selecionar disciplina, prioridade Alta e data limite.

3. Clicar em "Salvar Atividade".

- **Resultado Esperado:** A tarefa é registrada com sucesso com status inicial "Pendente", nível de urgência calculado automaticamente e exibida na listagem.
- **Candidato à Automação:** Sim (Automatizado com Selenium).
- **Responsável:** João Lucas de Antonio Boconcelo

Caso CT-020 Criação de atividade com título de 2 caracteres (Limite inferior inválido)

- **Requisito Relacionado:** RF-04 | **Risco:** R-03
- **Nível/Tipo:** Sistema / Limite Negativo
- **Técnica Aplicada:** Análise de Valor Limite (BVA - Valor abaixo do mínimo de 3 caracteres)
- **Prioridade:** Média
- **Pré-condições:** Aplicação e banco fictício inicializados; para CT-011 em diante, usuário de demonstração autenticado. Restabelecer a base entre cenários.

- **Dados de Entrada:** Título: "AB" (2 caracteres), Prazo: "2026-11-30", Disciplina: Selecionada
- **Passos Numerados:**

1. Informar título com apenas 2 caracteres.

2. Clicar em "Salvar Atividade".

- **Resultado Esperado:** O sistema barra a operação e exibe "O título da tarefa deve ter no mínimo 3 caracteres.".
- **Candidato à Automação:** Sim.
- **Responsável:** Lucas Braga

Caso CT-021 Criação de atividade sem informar data limite de entrega

- **Requisito Relacionado:** RF-04 | **Risco:** R-07
- **Nível/Tipo:** Sistema / Funcional Negativo
- **Técnica Aplicada:** Particionamento de Equivalência (Campo Obrigatório Ausente)
- **Prioridade:** Alta
- **Pré-condições:** Aplicação e banco fictício inicializados; para CT-011 em diante, usuário de demonstração autenticado. Restabelecer a base entre cenários.
- **Dados de Entrada:** Título: "Atividade Sem Data", Prazo: Vazio `""`, Disciplina: Selecionada
- **Passos Numerados:**

1. Preencher título e disciplina, deixando o campo de data vazio.

2. Submeter formulário.

- **Resultado Esperado:** O sistema exige o preenchimento da data limite informando "A data limite de entrega é obrigatória.".
- **Candidato à Automação:** Sim.
- **Responsável:** Lucas Braga

Caso CT-022 Atualização de status da tarefa e recálculo da taxa de conclusão no Dashboard

- **Requisito Relacionado:** RF-05, R-06 | **Risco:** R-06
- **Nível/Tipo:** Sistema / Transição de Estados
- **Técnica Aplicada:** Teste de Transição de Estados (Pendente -> Concluída)
- **Prioridade:** Alta
- **Pré-condições:** Uma tarefa está em estado "Pendente" e a taxa de conclusão atual está visível no Dashboard.
- **Passos Numerados:**

1. Acessar a tela de `/tarefas`.

2. Localizar a tarefa, selecionar a opção "Concluída" no dropdown de status e clicar no botão salvar.

3. Navegar até o Dashboard (`/`).

- **Resultado Esperado:** O status da tarefa muda para "Concluída", a contagem de tarefas concluídas aumenta em +1 e a barra de progresso exibe a nova taxa percentual calculada corretamente.
- **Candidato à Automação:** Sim (Automatizado com Selenium).
- **Responsável:** João Lucas de Antonio Boconcelo

Caso CT-023 Filtragem combinada por status "Pendente" e prioridade "Alta"

- **Requisito Relacionado:** RF-06 | **Risco:** R-08
- **Nível/Tipo:** Sistema / Tabela de Decisão
- **Técnica Aplicada:** Tabela de Decisão (Combinação de Filtros)
- **Prioridade:** Média
- **Pré-condições:** Existem tarefas com diferentes status e prioridades cadastradas.

- **Passos Numerados:**

1. Acessar `/tarefas`.

2. No filtro de Status selecionar "Pendente".

3. No filtro de Prioridade selecionar "Alta".

4. Clicar no botão "Filtrar".

- **Resultado Esperado:** Todas as tarefas exibidas na tabela devem obrigatoriamente possuir status "Pendente" E prioridade "Alta". Nenhuma tarefa concluída ou de baixa prioridade deve aparecer.
- **Candidato à Automação:** Sim.
- **Responsável:** Gabriel Henrique Moreira

Caso CT-024 Busca textual por termo inexistente

- **Requisito Relacionado:** RF-06 | **Risco:** R-08
- **Nível/Tipo:** Sistema / Funcional Alternativo
- **Técnica Aplicada:** Particionamento de Equivalência (Busca sem correspondência)
- **Prioridade:** Baixa
- **Pré-condições:** Aplicação e banco fictício inicializados; para CT-011 em diante, usuário de demonstração autenticado. Restabelecer a base entre cenários.
- **Dados de Entrada:** Termo de busca: `XYZ999TERMOINEXISTENTE`
- **Passos Numerados:**

1. Digitar o termo inexistente no campo "Buscar por Palavra-Chave".

2. Clicar em "Filtrar".

- **Resultado Esperado:** A tabela não deve quebrar nem disparar erro 500; deve exibir uma linha informativa com a mensagem amigável: "Nenhuma atividade encontrada para os critérios selecionados.".
- **Candidato à Automação:** Sim.
- **Responsável:** Lucas Braga

Apêndice B Registros detalhados dos defeitos

DEF-03 Taxa de conclusão aceita concluídas positivas com total zero

Requisito: RF-05. Risco: R-06. Severidade: Média. Prioridade: Alta.

Ambiente: Python 3.13.2, Flask, SQLite local isolado. Data: 11/11/2026. Status: Encerrado na revisão local.

Obtido antes: calculate_completion_rate(0, 1) retornava 0.0 em vez de rejeitar a inconsistência.

Esperado: rejeitar entradas inconsistentes, preservar isolamento e responder sem erro interno.

Correção: Verificar completed_tasks > total_tasks antes do caso vazio.

Reprodução e reteste: `test_tdd_consistencia.py / red.xml e green.xml`. Linha de base: commit a0df9e6a2dd69687bc88222fca707bdc6f877583. Correção publicada: commit 33d441c7f6dc4599da735727f232c5c1aa5d41f3. Logs consolidados em `11_METRICAS_E_QUALIDADE/evidencias/testes.xml`.

Responsabilidade: revisão técnica; validação dos integrantes pendente.

DEF-04 Configuração de banco ignorada e testes contaminam demonstração

Requisito: RF-03/RF-04. Risco: R-05. Severidade: Média. Prioridade: Alta.

Ambiente: Python 3.13.2, Flask, SQLite local isolado. Data: 11/11/2026. Status: Encerrado na revisão local.

Obtido antes: models.py usava caminho global enquanto create_app aceitava DATABASE sem aplicá-lo.

Esperado: rejeitar entradas inconsistentes, preservar isolamento e responder sem erro interno.

Correção: Ler DATABASE de current_app, inicializar dentro do contexto e usar temporários.

Reprodução e reteste: `test_isolated_databases_do_not_share_records`. Linha de base: commit a0df9e6a2dd69687bc88222fca707bdc6f877583. Correção publicada: commit 33d441c7f6dc4599da735727f232c5c1aa5d41f3. Logs consolidados em `11_METRICAS_E_QUALIDADE/evidencias/testes.xml`.

Responsabilidade: revisão técnica; validação dos integrantes pendente.

DEF-05 API permite leitura e alteração sem sessão e mistura dados de usuários

Requisito: RF-02. Risco: R-02. Severidade: Alta. Prioridade: Alta.

Ambiente: Python 3.13.2, Flask, SQLite local isolado. Data: 11/11/2026. Status: Encerrado na revisão local.

Obtido antes: Endpoints não verificavam sessão e gravavam user_id=1. Consultas HTML não filtravam o proprietário.

Esperado: rejeitar entradas inconsistentes, preservar isolamento e responder sem erro interno.

Correção: Exigir sessão na API e filtrar leituras e escritas por user_id.

Reprodução e reteste: `test_r02_api_requires_session / test_r02_users_cannot_read_or_modify_each_others_records`. Linha de base: commit a0df9e6a2dd69687bc88222fca707bdc6f877583. Correção publicada: commit 33d441c7f6dc4599da735727f232c5c1aa5d41f3. Logs consolidados em `11_METRICAS_E_QUALIDADE/evidencias/testes.xml`.

Responsabilidade: revisão técnica; validação dos integrantes pendente.

DEF-06 Data inválida na API causa exceção em vez de resposta 400

Requisito: RF-04. Risco: R-07. Severidade: Alta. Prioridade: Alta.

Ambiente: Python 3.13.2, Flask, SQLite local isolado. Data: 11/11/2026. Status: Encerrado na revisão local.

Obtido antes: POST /api/tarefas com due_date=2026-02-30 propagava ValueError.

Esperado: rejeitar entradas inconsistentes, preservar isolamento e responder sem erro interno.

Correção: Capturar ValueError, encerrar conexão e retornar JSON com HTTP 400 sem persistência.

Reprodução e reteste: `test_rf04_invalid_task_does_not_persist`. Linha de base: commit a0df9e6a2dd69687bc88222fca707bdc6f877583. Correção publicada: commit 33d441c7f6dc4599da735727f232c5c1aa5d41f3. Logs consolidados em `11_METRICAS_E_QUALIDADE/evidencias/testes.xml`.

Responsabilidade: revisão técnica; validação dos integrantes pendente.

DEF-07 API recebe JSON ou credenciais de tipos inválidos

Requisito: RF-02. Risco: R-03. Severidade: Média. Prioridade: Alta. Status: corrigido e retestado local e remotamente; issue #6 encerrada após publicação.

Reprodução: enviar POST /api/auth/login com JSON [1], um escalar ou e-mail/senha não textuais. Antes, havia AttributeError ou erro interno. Esperado: HTTP 400 com erro JSON.

Correção: json_payload_error verifica o contrato antes do acesso a campos. O ciclo TDD novo registrou 17 falhas e três aprovações no RED, seguido de 20 aprovações no GREEN e no REFACTOR. Commits reais e logs estão em evidencias/tdd_json da pasta 12.

Reteste: test_tdd_validacao_json.py e suíte ampliada. Correção publicada no commit 4acc9d5d7c1d065e152ded6c19699e7c94057f49. A conclusão da nova execução remota é acompanhada separadamente.

DEF-08 Hash divergente do RNF-02 original

Requisito: RNF-02. Risco: R-01. Severidade: Média. Prioridade: Alta. Status: corrigido e retestado local e remotamente; issue #7 encerrada após publicação.

Antes: o cadastro usava scrypt por padrão, enquanto RNF-02 especificava PBKDF2-SHA256. A divergência é de requisito, sem declarar scrypt inseguro. Esperado: novos cadastros e seed em PBKDF2-SHA256 e compatibilidade com dados existentes.

Correção: método explícito no cadastro e seed, com atualização transparente do hash antigo após autenticação válida, preservando a senha. Reteste: test_hash_requisito.py cobre seed e migração por login web/API; o cadastro exige PBKDF2 no teste RNF-02. Commit: 4acc9d5d7c1d065e152ded6c19699e7c94057f49.

Vínculos: DEF-03 = issue #1; DEF-04 = #2; DEF-05 = #3; DEF-06 = #4; DEF-07 = #6; DEF-08 = #7. URLs completas e metadados estão no P07 e no diretório de evidências da pasta 06.

Apêndice C Evidências visuais selecionadas

As capturas completas da inspeção por caso ficam nas pastas 05 e 10. As figuras abaixo exemplificam autenticação, cadastro de disciplina, criação de tarefa e progresso no conjunto Selenium. Foram selecionadas as capturas desse conjunto para manter legibilidade no papel. São imagens reais do sistema no navegador, não reconstruções.

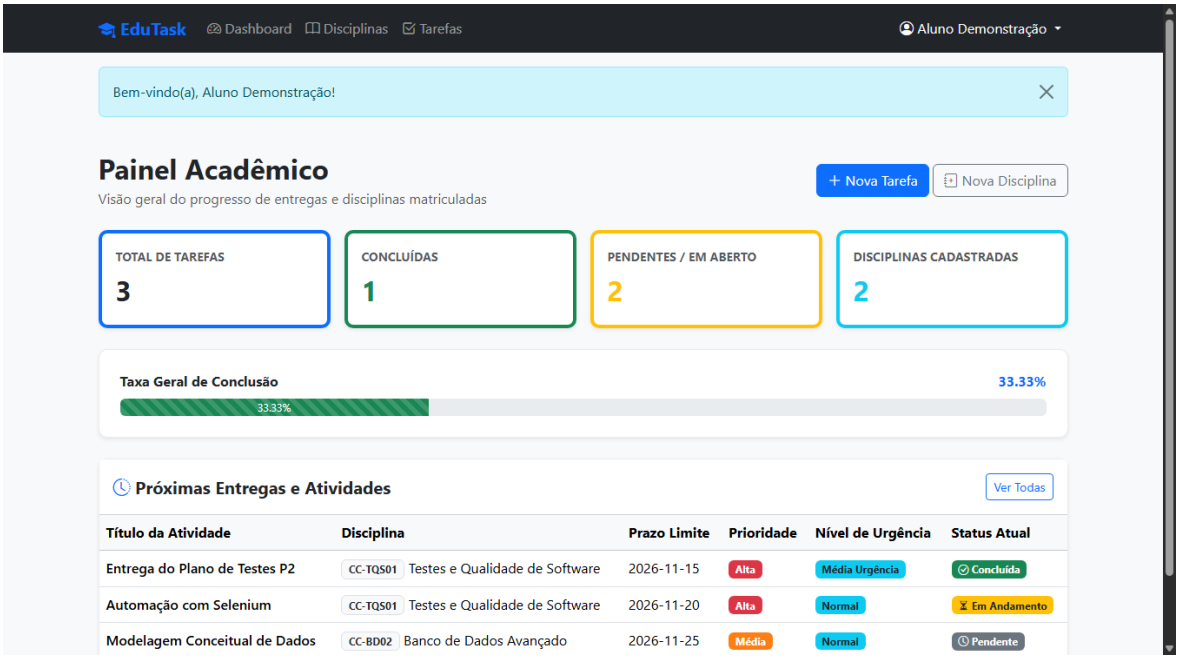


Figura 4 Painel após autenticação válida E2E 01

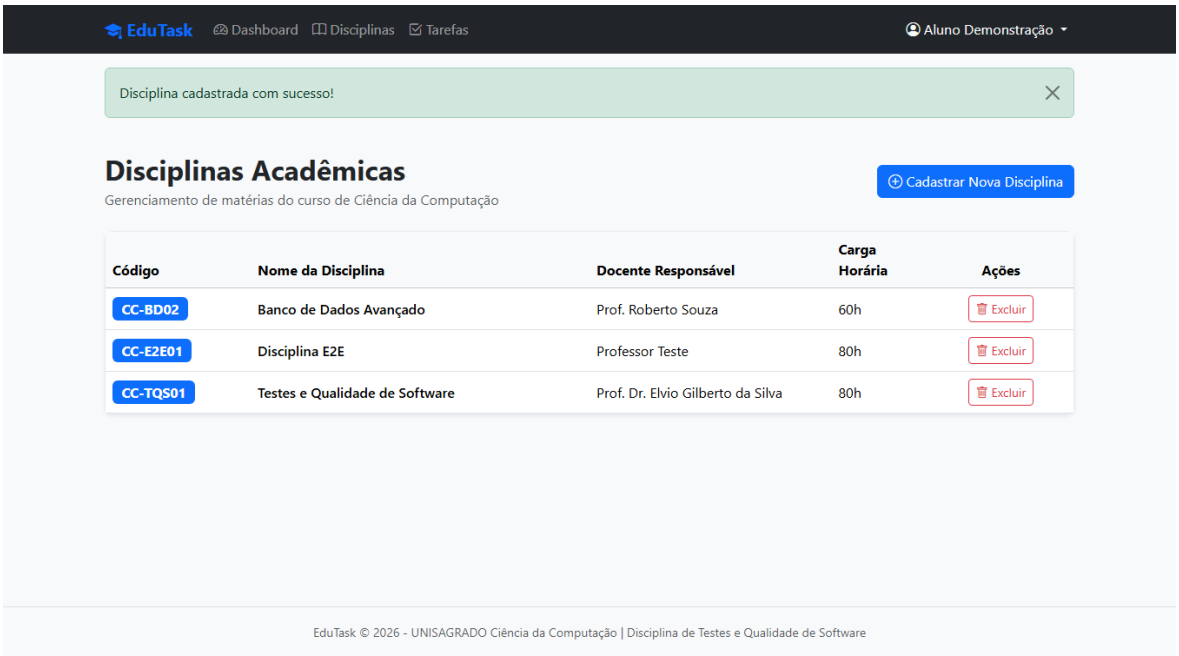


Figura 5 Disciplina cadastrada e conferida na lista E2E 02

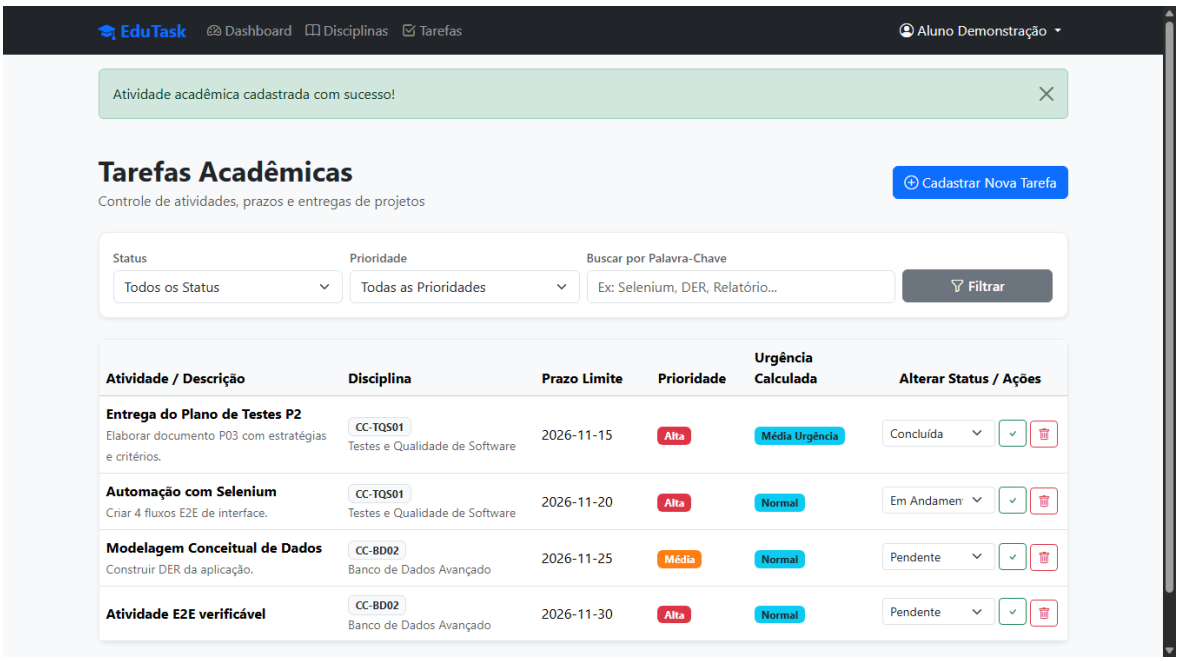


Figura 6 Tarefa criada pela interface E2E 03

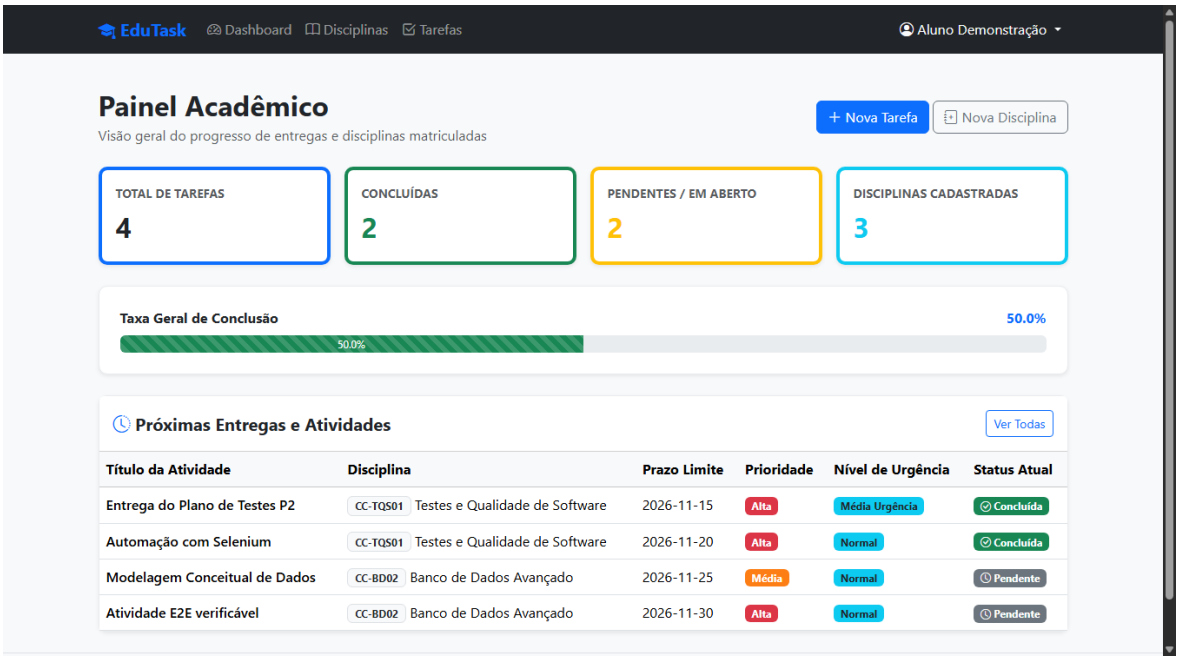


Figura 7 Percentual recalculado após concluir tarefa E2E 04

Apêndice D Instalação execução e organização da entrega

Ambiente e versões verificadas

Dependência	Versão local
Flask	3.1.3
Werkzeug	3.1.8
pytest	9.1.1
requests	2.34.2
selenium	4.51.0
webdriver-manager	4.1.2
python-docx	1.2.0
matplotlib	3.11.2
flake8	7.4.1
coverage	7.17.2

Os requisitos declaram versões mínimas e as versões verificadas estão registradas no JSON de ambiente. A versão do driver deve ser compatível com o Chrome instalado. O primeiro uso pode precisar de rede; os testes de API não precisam do navegador.

Instalação configuração e execução

```
python -m venv .venv
.venv/Scripts/Activate.ps1
python -m pip install -r 01_SISTEMA_ALVO/requirements.txt
python 01_SISTEMA_ALVO/run.py
```

Abrir http://127.0.0.1:5000 . Usuário fictício: aluno@unisagrado.edu.br. Senha fictícia: Senha@123. O banco de demonstração é criado automaticamente em 01_SISTEMA_ALVO/edutask.db. Não é necessário apagar esse banco para rodar os testes, que usam configurações isoladas.

O valor de SECRET_KEY pode ser definido no ambiente. Para reinicializar somente a demonstração, parar o servidor, remover apenas o arquivo edutask.db e reiniciar. Dados de usuários reais não devem ser usados no laboratório. Datas iniciais são fixas, enquanto o cálculo de urgência acompanha a data de consulta.

Comandos de verificação

```
python -m pytest -v 07_AUTOMACAO_UNITARIA
python -m pytest -v 08_INTEGRACAO_API
python -m pytest -v 05_EXECUCAO_MANUAL/test_reexecucao_catalogo.py
python -m pytest -v 09_SISTEMA_INTERFACE
python -m pytest -v 10_ACEITACAO_NAO_FUNCIONAIS
python tools/medir_qualidade.py
python tools/verificar_pipeline.py
```

Organização final das pastas

Pasta	Conteúdo necessário
00_IDENTIFICACAO_E_README	Identificação, ficha docente, instruções, confirmações restantes e minuta de entrega
01_SISTEMA_ALVO	Aplicação, templates, dependências e configuração de análise
02_REQUISITOS_ESCOPO_RISCOS	Oito requisitos e oito riscos priorizados

Pasta	Conteúdo necessário
03_PLANO_DE_TESTES	Estratégia, ambiente, dados, critérios e responsabilidades planejadas
04_CASOS_E_RASTREABILIDADE	Catálogo, matriz e CSV editável
05_EXECUCAO_MANUAL	Registro atual de inspeção, 24 capturas e reexecução automatizada
06_DEFEITOS	Seis defeitos, exportação e metadados das issues
07_AUTOMACAO_UNITARIA	Trinta cenários unitários
08_INTEGRACAO_API	Testes integrados, coleção e ambiente Postman
09_SISTEMA_INTERFACE	Quatro fluxos Selenium e suas evidências
10_ACEITACAO_NAO_FUNCIONAIS	Cenários, critérios não funcionais e quatro capturas de aceitação técnica
11_METRICAS_E_QUALIDADE	Planilha com fórmulas, gráficos, latências, cobertura e relatórios
12_TDD_CICD_MELHORIA	Ciclo TDD, logs, metadados e evidências do Actions
13_RELATORIO_FINAL	Relatório canônico completo em DOCX e PDF e fonte de geração
14_APRESENTACAO_E_VIDEO_CONTINGENCIA	Registro da dispensa, sem materiais antigos divergentes
15_DIARIO_DE_PARTICIPACAO	Atividades verificáveis e declaração para conferência/assinatura
16_BACKUP_ENTREGA	Checklist, manifesto SHA-256 e pacote de backup

Os utilitários de tools apoiam medição, reprodução e geração documental. Caches, bancos temporários, dependências instaladas localmente, cópias de relatório, slides dispensados e arquivos transitórios de publicação não fazem parte da entrega final.

Evidências e integridade

O manifesto SHA-256 identifica os conteúdos do pacote de backup, excluindo o próprio manifesto e o ZIP. O empacotador verifica a integridade do arquivo e compara os hashes de cada item. O ZIP não contém banco local, ambiente virtual ou credenciais. Os metadados de GitHub preservam IDs, commits e conclusões para consulta posterior.

Confirmações finais

Antes da entrega oficial, o grupo deve conferir a ficha docente, o atendimento do RNF-02 original, eventual licença de redistribuição, participação, assinaturas, dispensa de apresentação e permissões do Drive. O comprovante é uma minuta e deve receber data e hora reais somente após o envio. Nenhum desses atos foi antecipado no relatório.